

TECHNICAL PERFORMANCE REPORT

Efficient Ride-Sharing Simulator

Analysis of Event Processing, Driver Matching, Routing, and Service Performance

Harold L. R. Watkins

August 22, 2026

Python | Dijkstra's Algorithm | Quadtree | Discrete-Event Simulation

1. Introduction

This project models the operating core of a ride-sharing service as a deterministic, discrete-event simulation. The system coordinates a fleet of cars, dynamically generated rider requests, and a weighted road network while advancing directly from one meaningful event to the next. The principal goal was to determine whether efficient data structures could keep driver matching and route selection practical while preserving correct car and rider state throughout each trip.

The analysis focuses on two questions. First, can the system provide a favorable rider experience, measured through successful trip completion and pickup delay? Second, how efficiently does the fleet serve the generated demand, measured through trip duration and overall driver utilization? The results also show whether the chosen event queue, spatial index, and pathfinding strategy work together as a coherent simulation rather than as isolated algorithms.

2. Methodology

The architecture is organized around Simulation, Car, Rider, and Graph classes. A min-heap stores four-field events as (timestamp, sequence number, event type, data), allowing the engine to remove the next chronological event in $O(\log M)$ time for M pending events. A unique sequence number preserves deterministic ordering when timestamps match. This design is more efficient and reliable than repeatedly scanning or sorting an unsorted event list.

The city is stored as a weighted adjacency list. This representation requires $O(V + E)$ space and supports efficient neighbor traversal on a sparse road network, unlike an adjacency matrix that requires $O(V^2)$ storage. Dijkstra's algorithm computes the lowest-travel-time path because edge weights are nonnegative and therefore cannot be handled correctly by an unweighted breadth-first search. With a binary heap, routing requires $O((V + E) \log V)$ time.

Available cars are indexed in a Quadtree rather than examined by a full $O(N)$ fleet scan. For each request, the Quadtree returns up to five geographically near candidates; Dijkstra then evaluates the true road-network travel time for each candidate, and the best reachable car is dispatched. This hybrid strategy reduces the number of expensive route calculations while improving accuracy over selecting a driver by straight-line distance alone.

Run Parameter	Analyzed Configuration
Rider-generation window	200 simulation time units
Demand and fleet	25-rider cap; 100 cars
Request process	Mean interarrival time 8; random seed 549
Matching	Five Quadtree candidates evaluated with Dijkstra
Map and output	city_map.csv; simulation_summary.png

3. Results

The run generated 18 requests before the 200-time-unit generation limit was reached. All 18 riders completed their trips, and the event queue continued processing active pickup and drop-off events until the final simulation time of 341.02. The run performed 108 Dijkstra evaluations and produced no unmatched or unsuccessful riders.

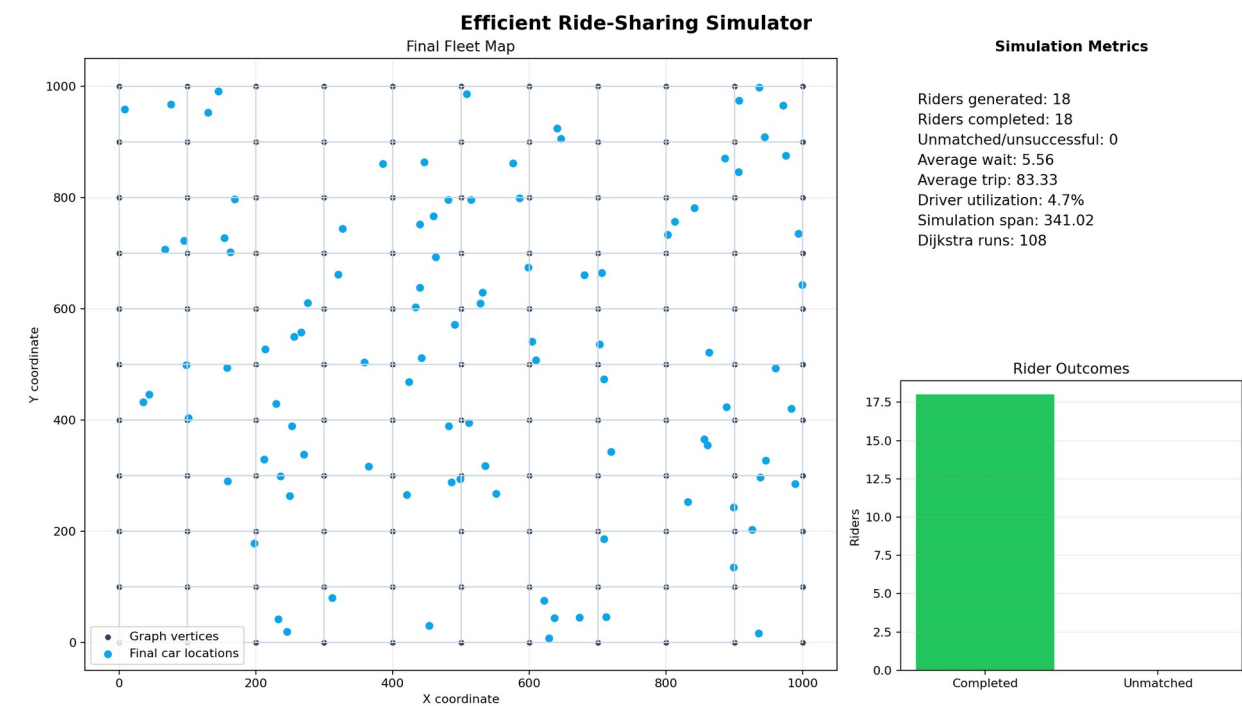


Figure 1. Integrated final fleet map, simulation metrics, and rider outcomes generated by the program.

Key Performance Indicator	Result	Operational Meaning
Total completed trips	18	All generated requests completed
Average rider wait	5.56 time units	Short delay from request to pickup
Average trip duration	83.33 time units	Network-weighted passenger travel
Overall driver utilization	4.7%	Fleet capacity substantially exceeded demand

4. Discussion

The results indicate a reliable and rider-friendly service under the analyzed configuration. Every generated request completed successfully, and the average wait was 5.56 time units compared with an average in-vehicle trip of 83.33 time units. The pickup delay was therefore small relative to the typical trip duration. The zero-unmatched result also shows that the fleet and routing logic maintained enough reachable capacity throughout the run.

However, the 4.7% utilization rate reveals that this performance was achieved with substantial excess capacity. One hundred cars served only 18 trips, so most drivers remained idle for most of the 341.02-time-unit simulation span. This exposes the central trade-off: a large fleet improves geographic coverage and reduces rider wait, but it lowers asset utilization. A smaller fleet would likely increase utilization and operating efficiency, while also increasing the risk of longer waits or unmatched requests during overlapping demand.

Hypothesis. Reducing the fleet from 100 cars to 25 or 50 cars while holding the request process constant would materially increase driver utilization. Average rider wait would probably rise because fewer cars would be positioned near each request, but the increase may remain moderate until demand begins to overlap more heavily. A parameter sweep across several fleet sizes and random seeds would identify the point where improved utilization begins to cause unacceptable wait times or unsuccessful matches.

The model also has important limitations. Road weights remain constant, so traffic, incidents, weather, construction, and time-of-day variation are omitted. Random requests are uniformly distributed rather than concentrated around realistic neighborhoods or activity centers. Cars carry one rider at a time, and the model does not include cancellations, ride pooling, driver breaks, pricing, fuel or charging constraints, or changes in fleet availability. Arbitrary coordinates are snapped to the nearest graph vertex, which simplifies routing but can distort first-mile travel. Finally, this report analyzes one reproducible run, so the results establish deterministic behavior rather than statistical confidence across many demand patterns.

5. Conclusion

The integrated simulator successfully completed every generated trip with short pickup delays, but its low driver utilization shows that the analyzed fleet was overprovisioned for the demand. The system's min-heap, adjacency-list graph, Dijkstra routing, and Quadtree candidate search provided a clean and efficient foundation for the simulation.

Future work should run repeated scenario experiments across fleet sizes and arrival rates, then report distributions and confidence intervals for wait time, completion rate, and utilization. Time-dependent road weights and demand hotspots would further improve realism and allow the simulator to evaluate congestion-aware routing, fleet repositioning, and surge-pricing strategies.